Department of Computer Science & Engineering
The University of Texas at Arlington

Architectural Design Specification
CSE 4317: Senior Design II
Spring 2025

MindMeal
MindMeal(ADHD Food Planner)

Rikita Karanjit
Saina Shrestha
Mohammad Hasibur Rahman
Rakshya Basnet
Utshah Neupane
Saul Mora

Revision History

Revision	Date	Author(s)	Description
0.1	08.30.2025	RK	document creation
0.2	08.28.2025	MHR, RK, SS	Introduction Complete
0.3	09.02.2025	MHR, RK, SS	System overview complete
1.0	09.03.2025	UN, RB, SM	First draft
1.1	09.04.2025	MHR, RB, RK, SM, SS, UN	Team Review
1.2	09.10.2025	MHR, RB, RK, SM, SS, UN	Overall Review and Document Completion
1.3	09.19.2025	RK	Changed the Logo
1.4	09.19.2025	MHR, RB, RK, SM, SS, UN	Review
1.5	12.07.2025	MHR, RB, RK, UN	Updated Logo and final review

CONTENTS

1 Introduction **5**
- 1.1 Product Concept . . . 5
- 1.2 Scope . . . 5
- 1.3 Key Requirements . . . 6

2 System Overview **7**
- 2.1 Architecture . . . 7
- 2.2 User Interface Components . . . 7
- 2.3 Application Logic . . . 8
- 2.4 Data Management . . . 10

3 Subsystem Definitions & Data Flow **11**
- 3.1 Module Definitions . . . 11
- 3.2 Data Flow Examples . . . 11
- 3.3 Integration Between Modules . . . 11

4 UI Layer Subsystems **12**
- 4.1 Login/Registration . . . 12
- 4.2 Onboarding . . . 13
- 4.3 Home Page . . . 14
- 4.4 Inventory . . . 15
- 4.5 Recipe Maker (Chat) . . . 16
- 4.6 Notifications . . . 17
- 4.7 Camera Module (Future) . . . 18

5 Application Logic Layer Subsystems **20**
- 5.1 Authentication Logic . . . 20
- 5.2 Onboarding Logic . . . 20
- 5.3 Recipe Engine . . . 21
- 5.4 Notification Logic . . . 22
- 5.5 Inventory Management Logic . . . 23
- 5.6 Personal Information Manager . . . 24

6 Data Layer Subsystems **26**
- 6.1 API Client . . . 26
- 6.2 Local Storage . . . 27
- 6.3 Push Notification Service . . . 28
- 6.4 External Integrations . . . 29

List of Figures

1 Mobile app product concept for ADHD dietary management (MindMeal prototype). . . . 5
2 High-level architecture of the ADHD dietary management app 7
3 User interface components of the app 8
4 Application logic diagram of the MindMeal 9
5 Data flow between core modules of the app 12
6 Application logic diagram of the MindMeal 13
7 Onboarding interface module 14
8 Home Page interface module 15
9 Inventory interface module 16
10 Recipe Maker (chat interface) module 17
11 Notifications interface module 18
12 Future Camera Module interface 19
13 Authentication logic workflow 20
14 Onboarding logic module2 21
15 Recipe engine and suggestion workflow 22
16 Notification logic flow 23
17 Inventory management logic module 24
18 Personal information management logic 25
19 API client and external communication flow 26
20 Local storage module for offline-first design 27
21 Push notification delivery system 28

List of Tables

1 Introduction

1.1 Product Concept

This project introduces a mobile application designed specifically to support individuals with ADHD in managing their diet and daily food routines. The app provides recipe suggestions based on the user's available inventory, allergies, dietary goals, and medical conditions. By combining intuitive design with smart suggestions, it helps users make healthier food choices without feeling overwhelmed.

Key features include:

- Personalized recipe recommendations based on inventory and health needs.
- Easy-to-use inventory management with options to add, update, and delete items.
- Notifications and reminders for expiring items, meal suggestions, and personal reminders.
- A friendly onboarding flow that collects user details such as age, allergies, medical conditions, and goals.

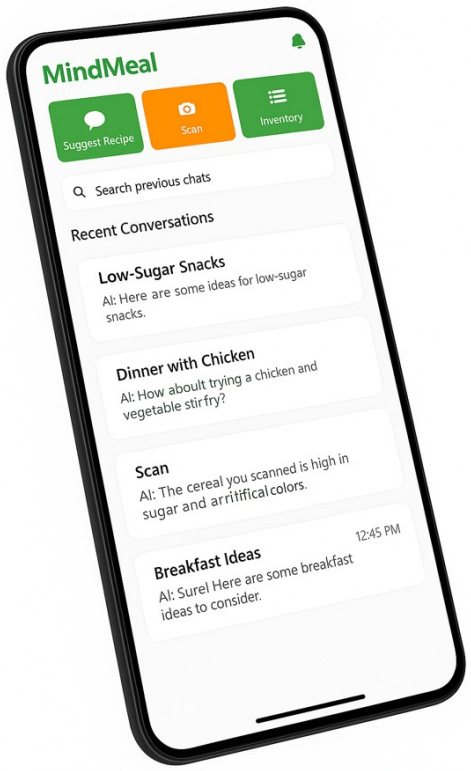

Figure 1: Mobile app product concept for ADHD dietary management (MindMeal prototype).

1.2 Scope

The app is built as a mobile-first solution using React Native and Material UI, ensuring a smooth and consistent experience across devices. In the first version, the app includes onboarding, recipe suggestions, inventory management, and notifications. Future enhancements, such as camera-based barcode scanning and nutrition database integration, will extend its capabilities.

The scope is designed to remain practical for daily use while being flexible enough to grow with new features as users' needs evolve.

1.3 Key Requirements

To ensure the app meets its objectives, the following requirements are central to its design and development:

- Personalized recipe suggestions that account for allergies, medical conditions, and dietary goals.
- A user interface that is simple, clear, and ADHD-friendly, reducing distractions and confusion.
- Inventory management functionality to add, update, and remove food items efficiently.
- Notifications for expiring inventory items, reminders, and contextual recipe prompts.

These requirements ensure the app remains useful, engaging, and reliable in helping users maintain healthier food habits.

2 System Overview

2.1 Architecture

The app architecture is structured in layers that separate the user interface, application logic, and data management. This approach makes the system modular, easier to maintain, and scalable for future updates. Each layer is responsible for specific tasks and communicates with others through well-defined interfaces.

The system includes:

- **UI Components**: Login/Registration, Onboarding, Home Page, Inventory, Recipe Maker (Chat), Notifications, Camera Module (future).
- **Application Logic**: Recipe engine, user profile handling, notification logic, and inventory rules.
- **Data Management**: Local storage (SQLite/AsyncStorage), API communication with external services and LLMs, and notification delivery.

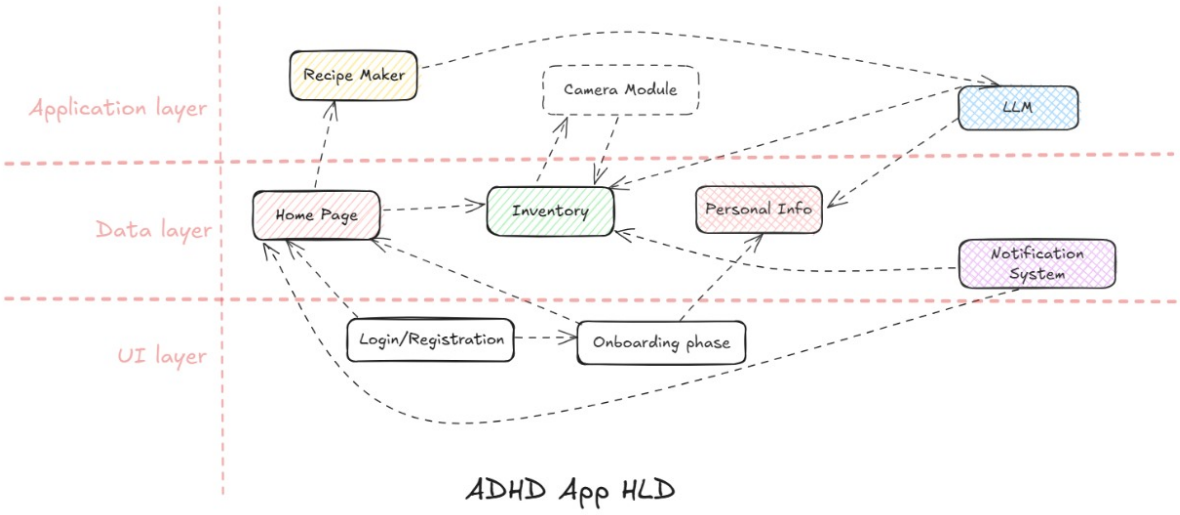

Figure 2: High-level architecture of the ADHD dietary management app

2.2 User Interface Components

The user interface is designed to be ADHD-friendly, with a focus on clarity and simplicity. Each component allows users to easily navigate and perform key actions. For example, the *Onboarding* module collects health and dietary information, the *Home Page* acts as a dashboard, the *Inventory* tracks available food, and the *Recipe Maker* provides personalized suggestions.

The interface also includes a notification system that displays reminders and alerts in a non-intrusive way, and a future *Camera Module* for scanning items.

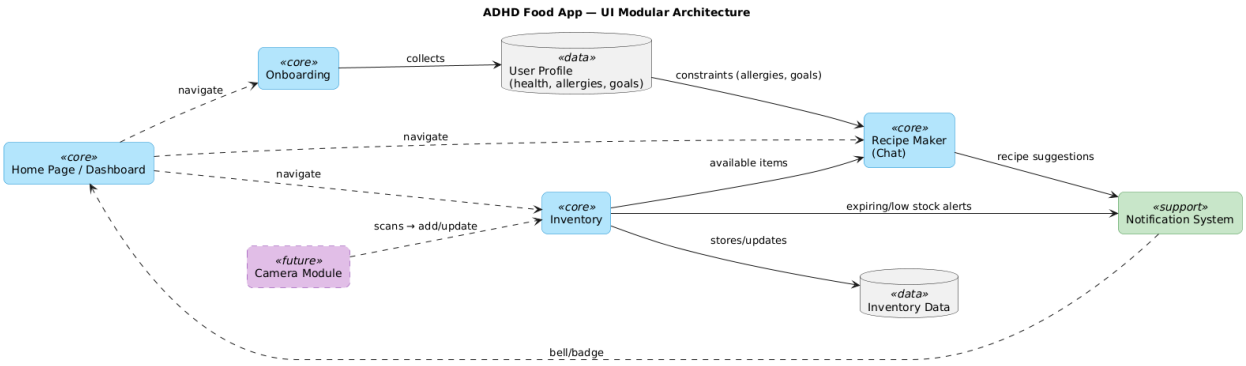

Figure 3: User interface components of the app

2.3 Application Logic

The application logic forms the "brain" of the app. It ensures that user data, inventory, and recipes are processed correctly. The logic handles workflows such as:

- Collecting onboarding data and updating the user profile.
- Processing recipe requests with inventory and dietary constraints.
- Managing inventory rules such as item addition, update, or deletion.
- Triggering reminders and notifications for expiring items or meal prompts.

This logical layer ensures that the app remains functional and personalized for each user.

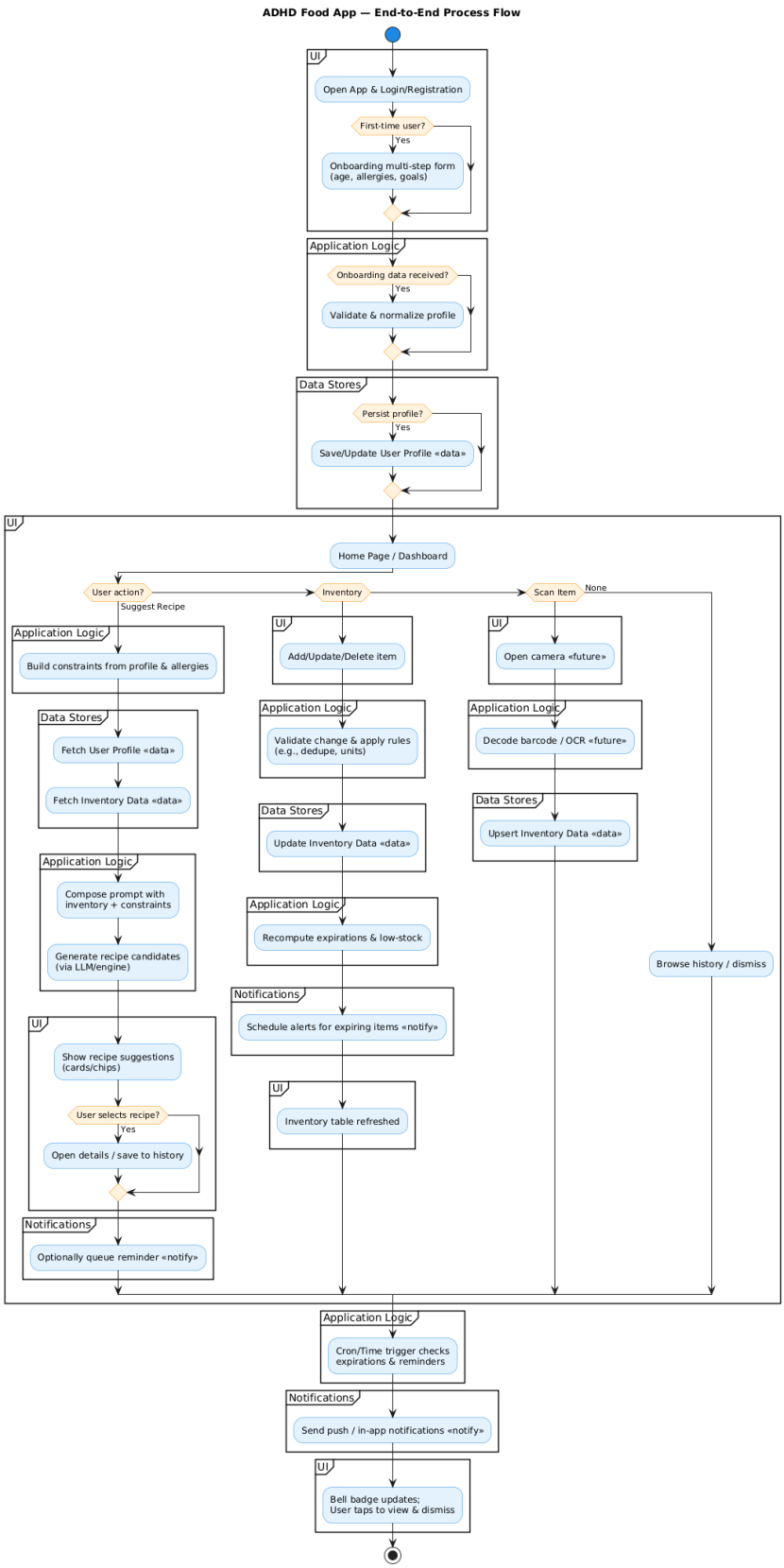

Figure 4: Application logic diagram of the MindMeal

2.4 Data Management

The data layer ensures that information is stored securely, synchronized properly, and retrieved efficiently. Local storage (SQLite/AsyncStorage) maintains inventory and user preferences offline, while API clients handle communication with external services and the recipe LLM. The push notification service ensures timely delivery of alerts and reminders.

Future integrations, such as barcode scanning and nutrition databases, will expand the ability of the app to provide even more accurate and relevant information.

3 Subsystem Definitions & Data Flow

3.1 Module Definitions

The app is divided into several modules, each with a clear responsibility. This modular design makes the system easy to understand, maintain, and expand in the future. Each module interacts with others through defined inputs and outputs, ensuring a smooth flow of information.

The main modules are:

- **Login/Registration**: Handles user authentication and account creation.
- **Onboarding**: Collects personal and health-related details such as allergies, conditions, and dietary goals.
- **Home Page**: Acts as a dashboard, linking users to recipe suggestions, inventory, and notifications.
- **Inventory**: Tracks food items with add, update, and delete options.
- **Recipe Maker (Chat)**: Generates recipe suggestions using the userâs inventory and preferences.
- **Notifications**: Displays alerts for expiring items, reminders, and recipe prompts.
- **Camera Module (Future)**: Planned feature for barcode and label scanning.

3.2 Data Flow Examples

The data flow in the app shows how information moves between modules and how different components interact. Each process begins with a user action and ends with a meaningful response or system update. Below are the main flows:

- **Login â Home â Inventory Sync**: After logging in, the app loads the home dashboard and synchronizes inventory data from local storage.
- **Onboarding â Personal Info â Recipe Maker**: Data collected during onboarding feeds directly into the recipe suggestion engine.
- **Inventory Updates â Notification System**: When an item is added or marked as expiring soon, the notification system is triggered.
- **Recipe Maker â LLM â Recipe Suggestions â Display on Home**: A user request is sent to the recipe engine, processed by the LLM, and the result is displayed as a recipe card or chat response.

3.3 Integration Between Modules

The modules are not isolated â they work together to provide a seamless user experience. For example, the *Recipe Maker* relies on both the *Inventory* and *Onboarding* modules to generate relevant suggestions. Similarly, the *Notification System* integrates with inventory and recipes to deliver timely reminders.

This integration ensures the app feels consistent and supportive, reducing friction for users with ADHD.

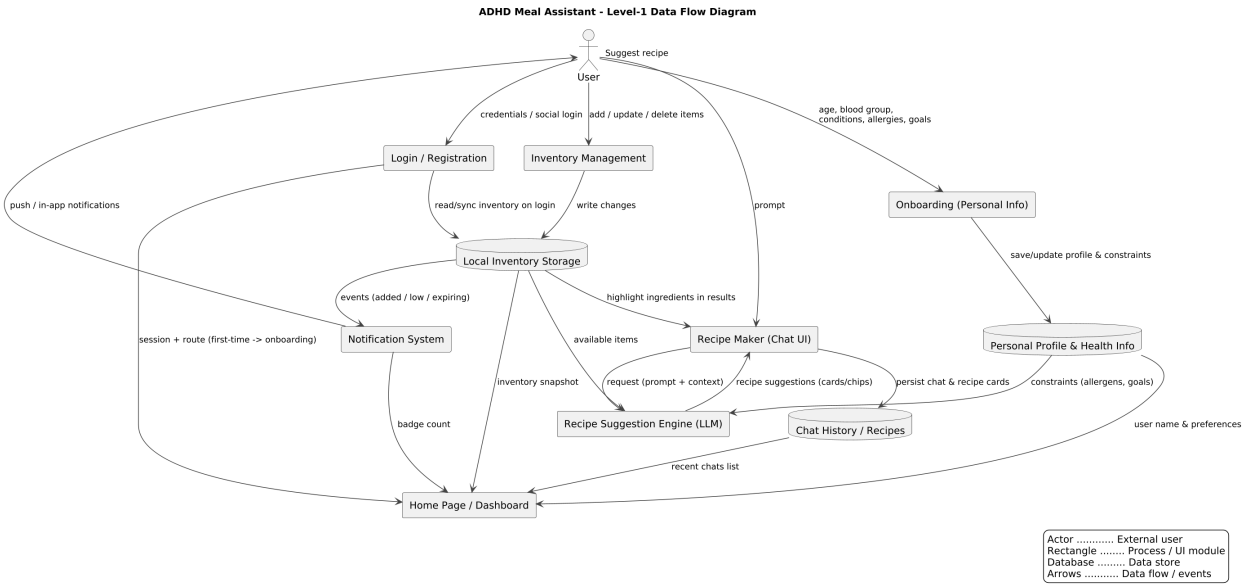

Figure 5: Data flow between core modules of the app

4 UI Layer Subsystems

4.1 Login/Registration

The Login and Registration screens provide secure access to the app. They are designed to be simple, distraction-free, and accessible for ADHD users. Clear labels, visibility toggles, and optional social login help reduce cognitive load.

Assumptions:

- Built using Material UI components with React Native.
- Secure backend authentication system.

Responsibilities:

- Handle login, registration, and password reset.
- Redirect users to Home or Onboarding (for first-time users).

Interfaces:

- User input: email, password, registration details.
- API calls for authentication and account creation.

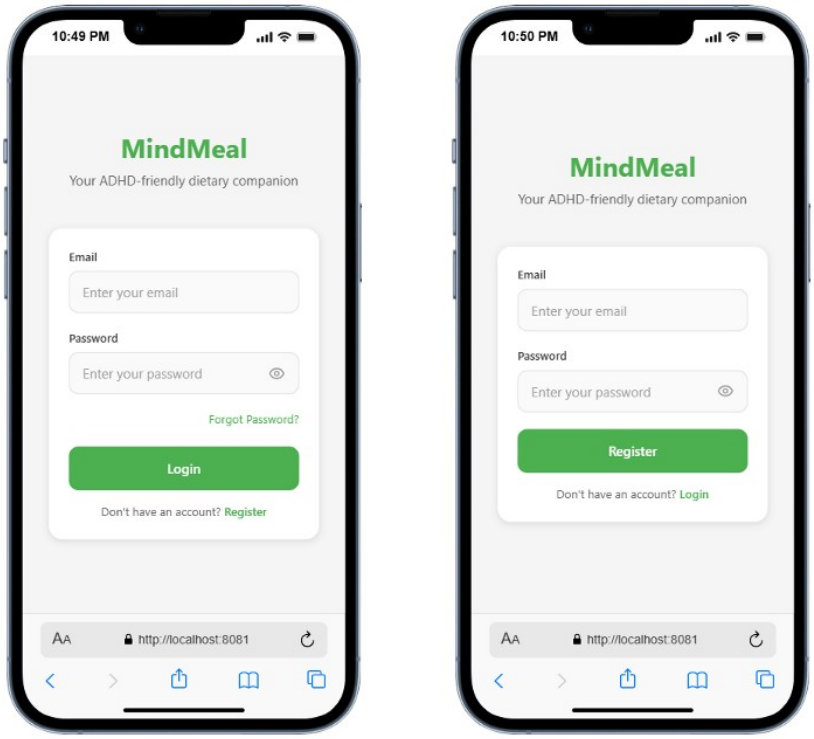

Figure 6: Application logic diagram of the MindMeal

4.2 ONBOARDING

The onboarding process collects essential personal and health data to personalize recipe suggestions. It uses a multi-step form with progress indicators to avoid overwhelming the user.

Assumptions:

- Simple step-by-step navigation with clear feedback.
- Data stored locally and synced with backend.

Responsibilities:

- Collect personal data (age, medical conditions, allergies, goals).
- Save data for use in recipe generation and notifications.

Interfaces:

- User input forms with text fields, checkboxes, and select menus.
- Storage APIs to save preferences.

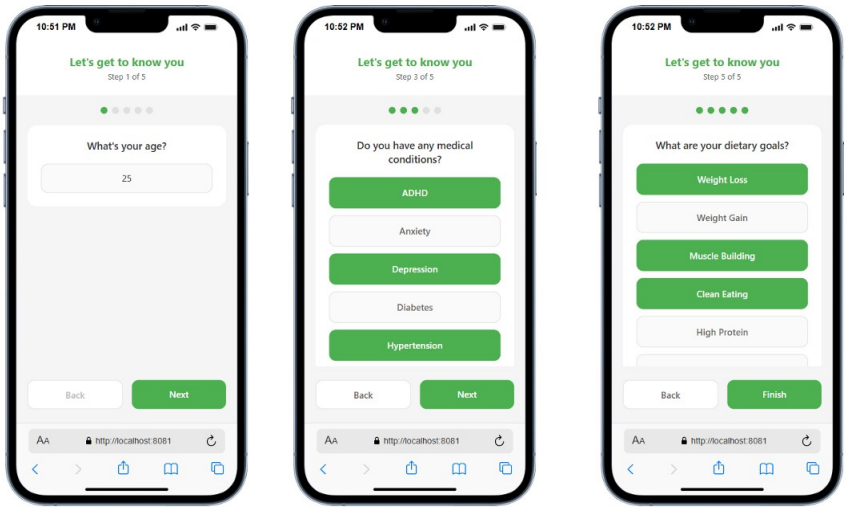

Figure 7: Onboarding interface module

4.3 Home Page

The Home Page acts as the central hub of the application. It links to key features such as recipe suggestions, inventory, and notifications. The interface is ADHD-friendly with a simple layout, clear navigation, and quick access buttons.

Assumptions:

- Dashboard layout with Material UI components.
- Search bar and notification icon included.

Responsibilities:

- Provide navigation to main modules.
- Display recent recipe chats and notifications.

Interfaces:

- Buttons to open Recipe Maker, Inventory, and Camera.
- Display table of previous recipe chats.

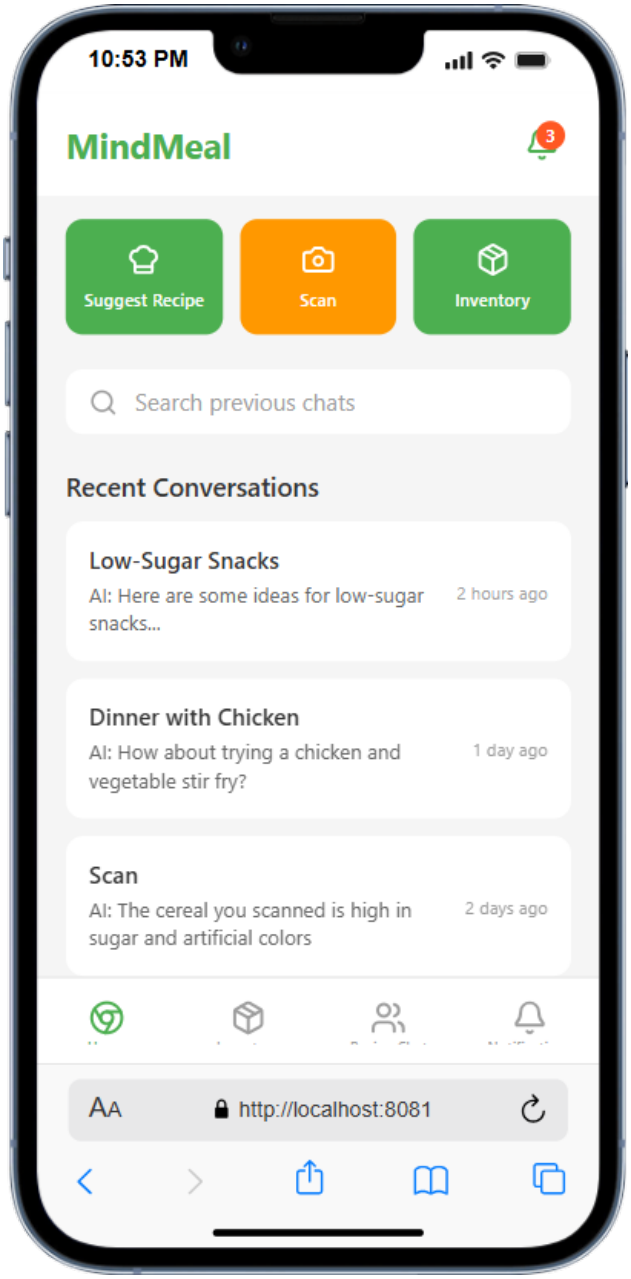

Figure 8: Home Page interface module

4.4 Inventory

The Inventory page lets users track available food items. It is designed to be quick, clear, and editable with minimal clicks. Floating action buttons simplify adding or updating items.

Assumptions:

- Local storage (SQLite/AsyncStorage) sync with backend.
- Simple add/update/delete functionality.

Responsibilities:

- Maintain food list with quantities and dates.
- Provide alerts for expiring items.

Interfaces:

- Add Item modal for user input.
- Table display with update and delete actions.

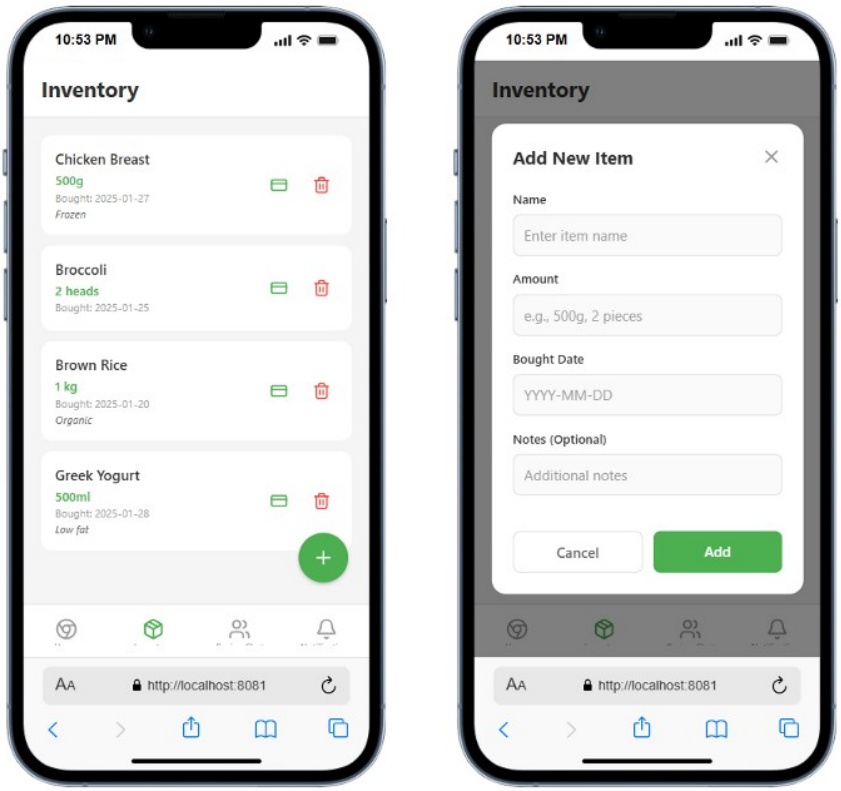

Figure 9: Inventory interface module

4.5 Recipe Maker (Chat)

The Recipe Maker is a conversational interface where users can request meal suggestions based on inventory and health needs. Recipes are displayed with prep time, ingredients, and allergen information.

Assumptions:

- Integrated with LLM for recipe generation.
- Chat-style interface using Material UI components.

Responsibilities:

- Accept user prompts for meal suggestions.
- Display recipe cards with ingredient highlighting.

Interfaces:

- Input text field for user queries.
- API calls to recipe engine and LLM service.

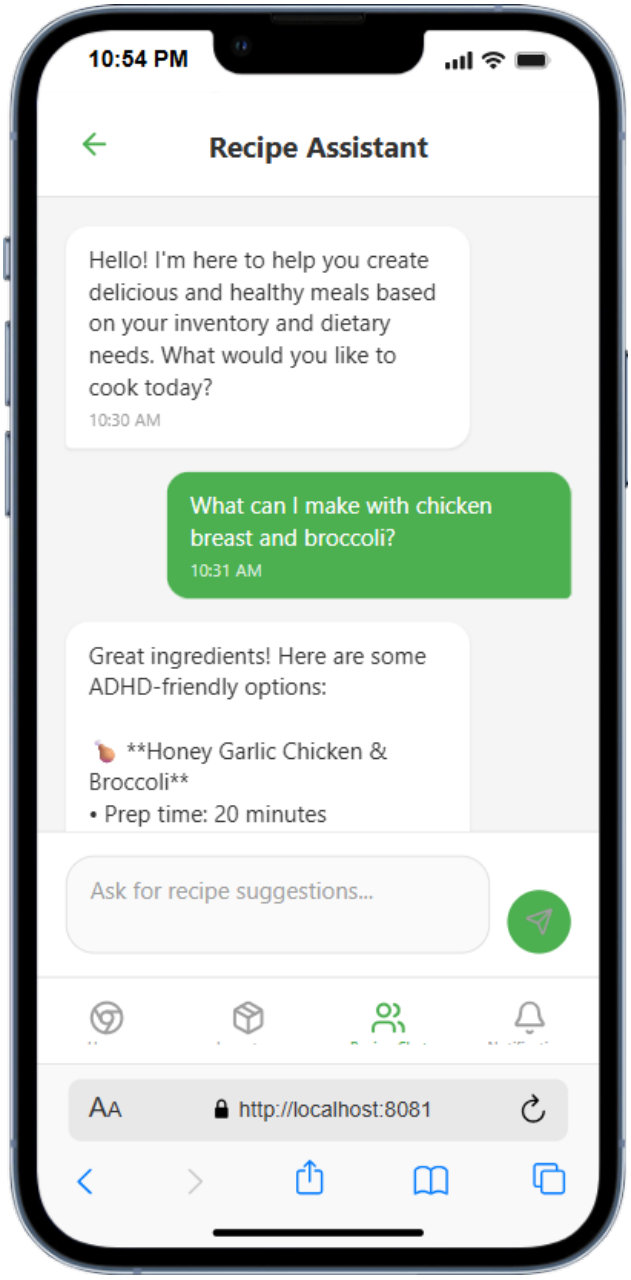

Figure 10: Recipe Maker (chat interface) module

4.6 Notifications

The Notifications module alerts users to expiring inventory, recipe prompts, or personal reminders. It groups alerts into categories for clarity and uses non-intrusive icons to avoid overwhelming users.

Assumptions:

- Push notification service integrated with app logic.
- Tabbed interface for different notification categories.

Responsibilities:

- Display alerts in categories (inventory, recipes, custom reminders).
- Allow users to dismiss or review alerts.

Interfaces:

- Notification bell icon on Home Page.
- Swipe-to-dismiss and tab navigation options.

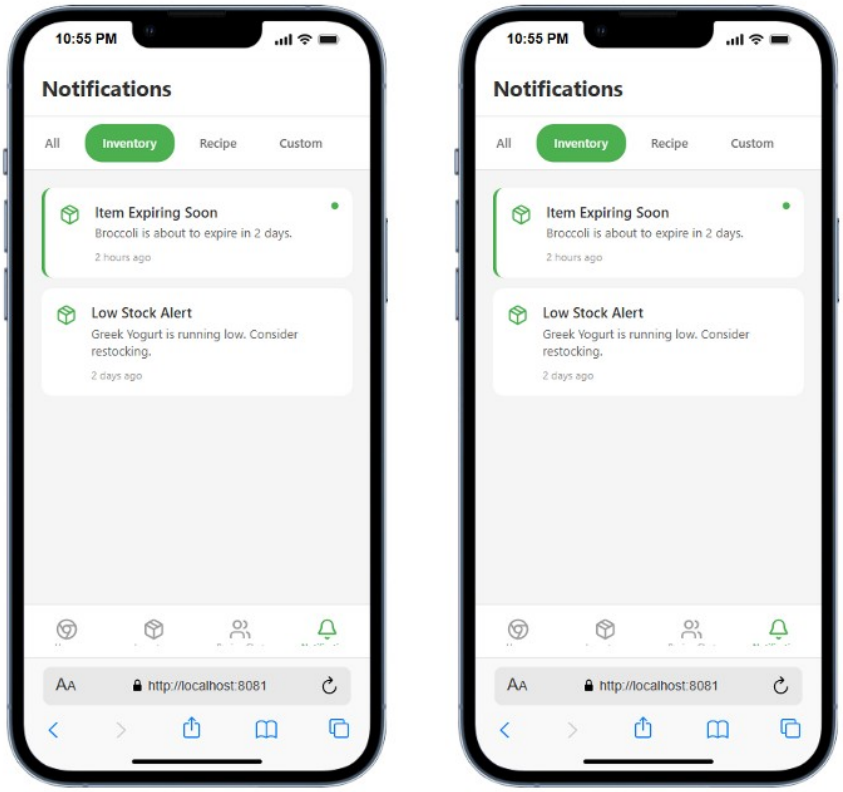

Figure 11: Notifications interface module

4.7 Camera Module (Future)

The Camera Module is planned for future versions. It will allow barcode and label scanning to quickly add food items to the inventory. Until implemented, it will display a simple placeholder screen.

Assumptions:

- Requires camera access permissions.
- Future integration with barcode/nutrition database.

Responsibilities:

- Provide quick scanning of food items.
- Automatically fill inventory fields with scanned data.

Interfaces:

- Camera viewfinder interface.
- API calls to barcode/nutrition database (future).

Figure 12: Future Camera Module interface

5 Application Logic Layer Subsystems

5.1 Authentication Logic

The authentication logic ensures secure access to the app. It verifies user credentials, manages sessions, and handles account creation. This module provides the foundation for protecting personal and health-related data.

Assumptions:

- Secure backend with encrypted password storage.
- Social login may be included as an optional feature.

Responsibilities:

- Validate login and registration requests.
- Manage sessions and token refresh.
- Handle password reset workflows.

Interfaces:

- Inputs: user credentials (email, password).
- Outputs: session tokens, login success/failure messages.

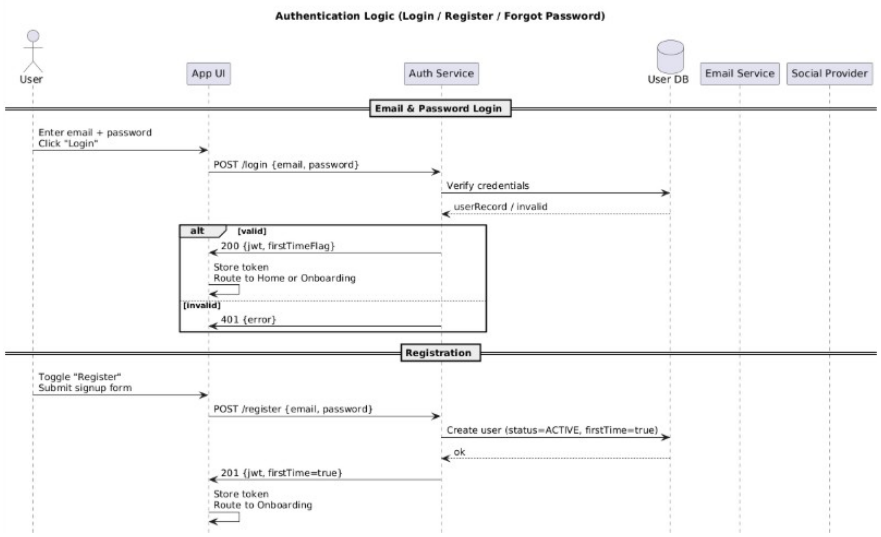

Figure 13: Authentication logic workflow

5.2 Onboarding Logic

The onboarding logic manages the flow of collecting personal data such as allergies, medical conditions, and dietary goals. It ensures the data is validated and stored for use in other modules.

Assumptions:

- Multi-step form reduces user overwhelm.

- Data is stored locally first and synced to the backend.

Responsibilities:

- Guide users through onboarding questions step by step.
- Validate input values and save them securely.
- Make personal data available to recipe and notification modules.

Interfaces:

- Inputs: age, blood group, allergies, conditions, goals.
- Outputs: structured personal profile data.

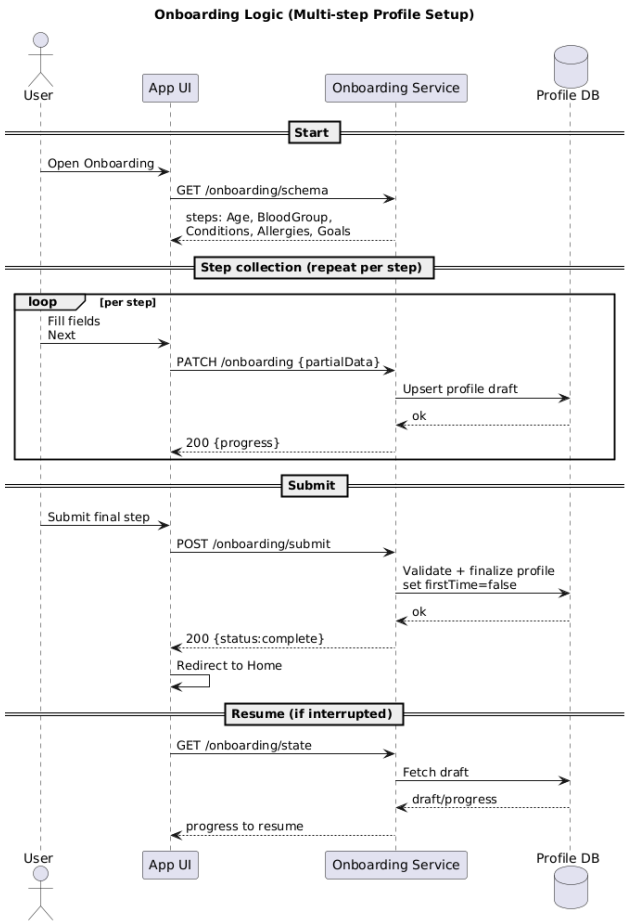

Figure 14: Onboarding logic module2

5.3 Recipe Engine

The recipe engine is the core intelligence of the app. It combines user preferences, inventory items, and external recipe databases/LLMs to generate meal suggestions.

Assumptions:

- Requires an internet connection for LLM-based suggestions.
- Must filter out allergens and respect dietary goals.

Responsibilities:

- Process user recipe requests.
- Match available inventory with recipe databases.
- Ensure allergen-free and goal-compliant suggestions.

Interfaces:

- Inputs: user query, inventory data, personal info.
- Outputs: structured recipe suggestions (title, ingredients, prep time).

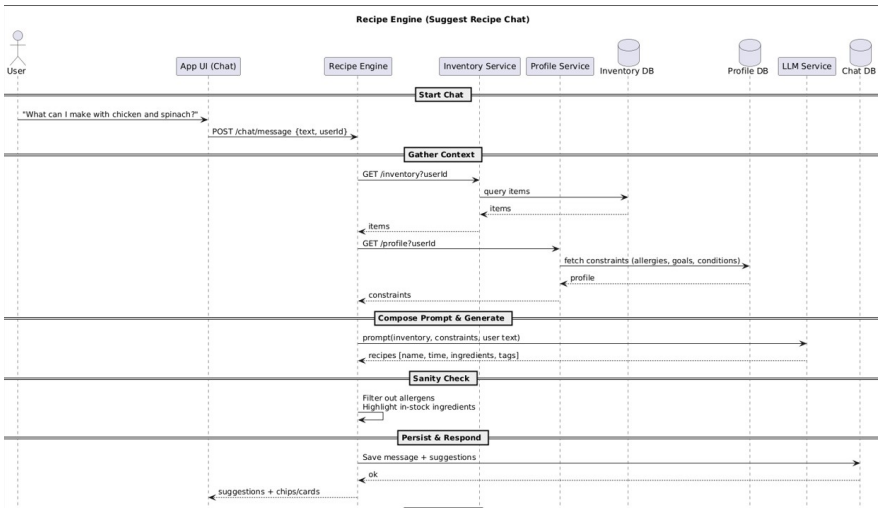

Figure 15: Recipe engine and suggestion workflow

5.4 Notification Logic

The notification logic manages reminders and alerts for users. It ensures that notifications are meaningful, timely, and ADHD-friendly (not overwhelming).

Assumptions:

- Push notifications are enabled on the device.
- Notifications are grouped by type (inventory, recipe, custom).

Responsibilities:

- Generate alerts for expiring items in the inventory.
- Send recipe reminders based on stock.
- Allow custom reminders (e.g., hydration, meal times).

Interfaces:

- Inputs: inventory updates, user preferences.
- Outputs: push notifications, in-app alerts.

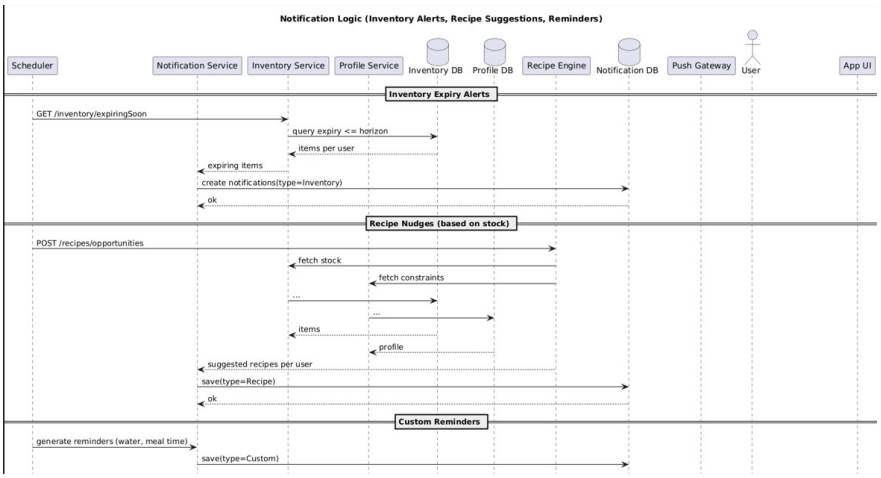

Figure 16: Notification logic flow

5.5 Inventory Management Logic

The inventory management logic keeps track of all food items. It ensures users can add, update, or remove items, and that inventory data is linked to recipes and notifications.

Assumptions:

- Inventory is stored in local storage and synced with cloud services.
- Users can manage items with minimal input steps.

Responsibilities:

- Add new items with details (name, amount, date).
- Update or delete items quickly.
- Provide real-time inventory data to recipe engine and notifications.

Interfaces:

- Inputs: user actions (add/update/delete).
- Outputs: updated inventory dataset.

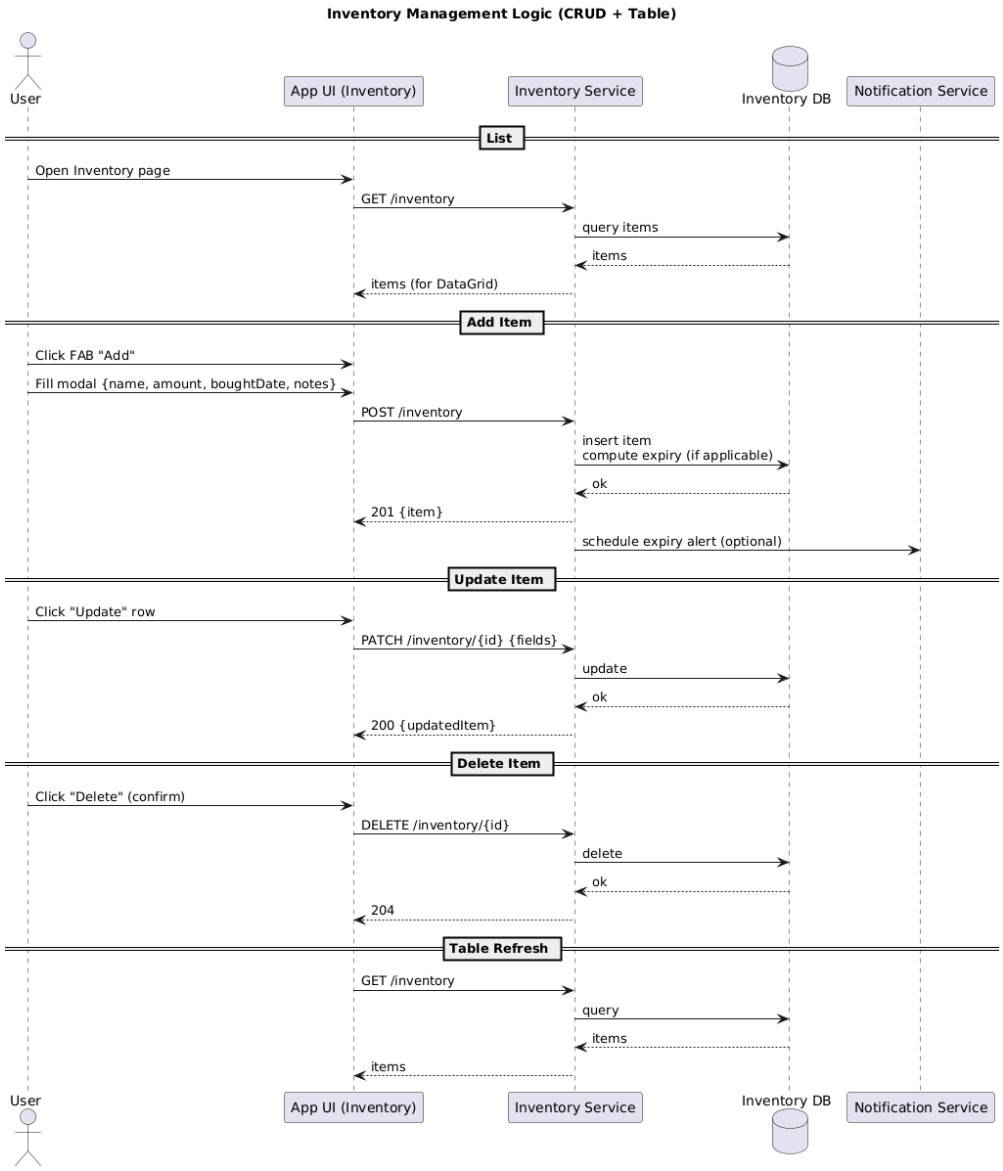

Figure 17: Inventory management logic module

5.6 Personal Information Manager

The personal information manager ensures all user-related details (medical conditions, allergies, goals) are consistently available across the app. It acts as a shared reference for personalization.

Assumptions:

- Data must be stored securely and accessed only with permissions.
- Must integrate seamlessly with onboarding, recipe engine, and notifications.

Responsibilities:

- Store and update personal information.
- Provide profile data to other logic modules.

- Ensure consistency across all app workflows.

Interfaces:

- Inputs: onboarding data, user updates.
- Outputs: structured personal profile accessible across modules.

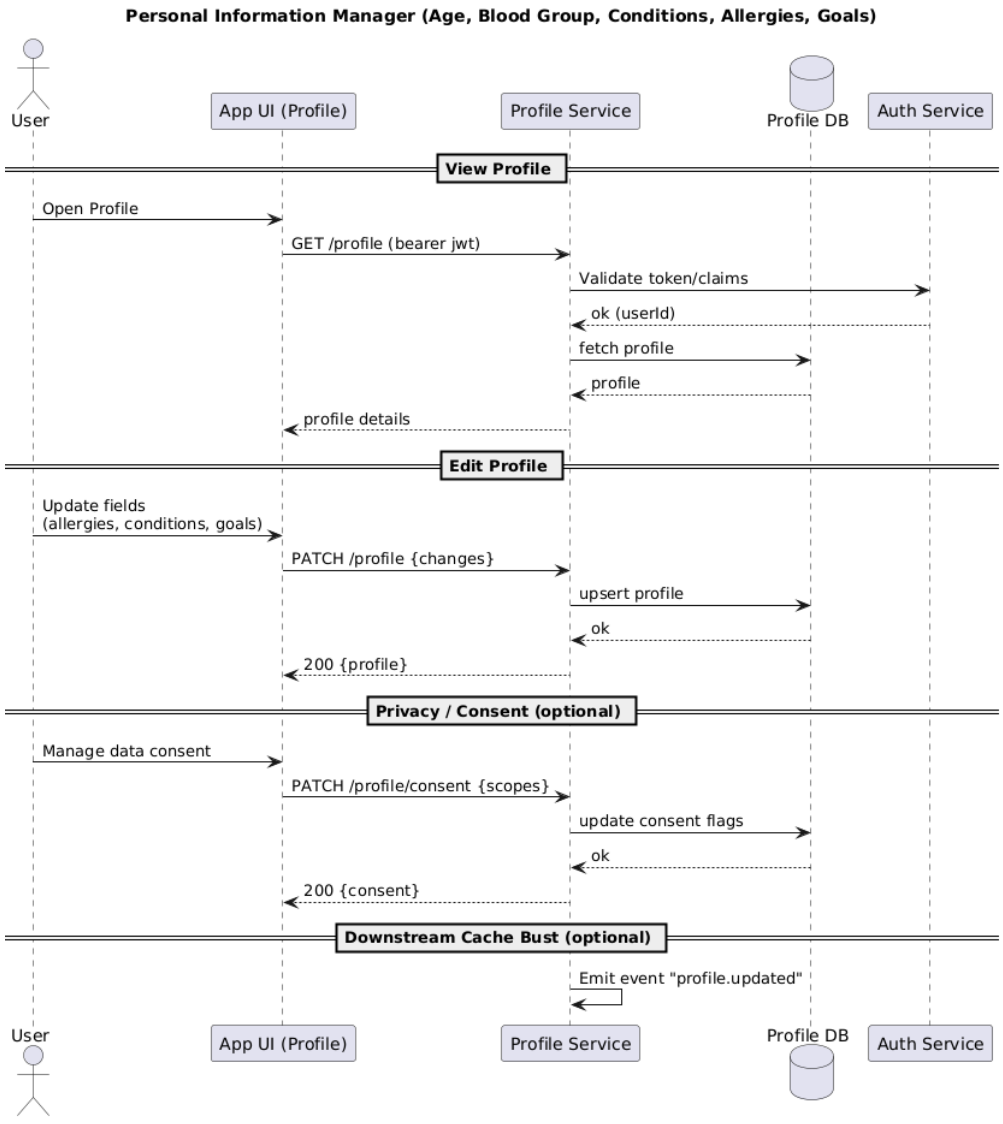

Figure 18: Personal information management logic

6 Data Layer Subsystems

6.1 API Client

The API Client handles communication between the mobile app, the recipe LLM, and external services. It ensures requests and responses are structured properly and passed to the right modules.

Assumptions:

- Requires stable internet connection.
- APIs are secured with authentication keys or tokens.

Responsibilities:

- Send user queries and inventory data to the recipe engine.
- Receive recipe suggestions and nutrition information.
- Handle error states and failed requests gracefully.

Interfaces:

- Inputs: inventory data, personal info, recipe requests.
- Outputs: recipe suggestions (JSON), nutrition facts.

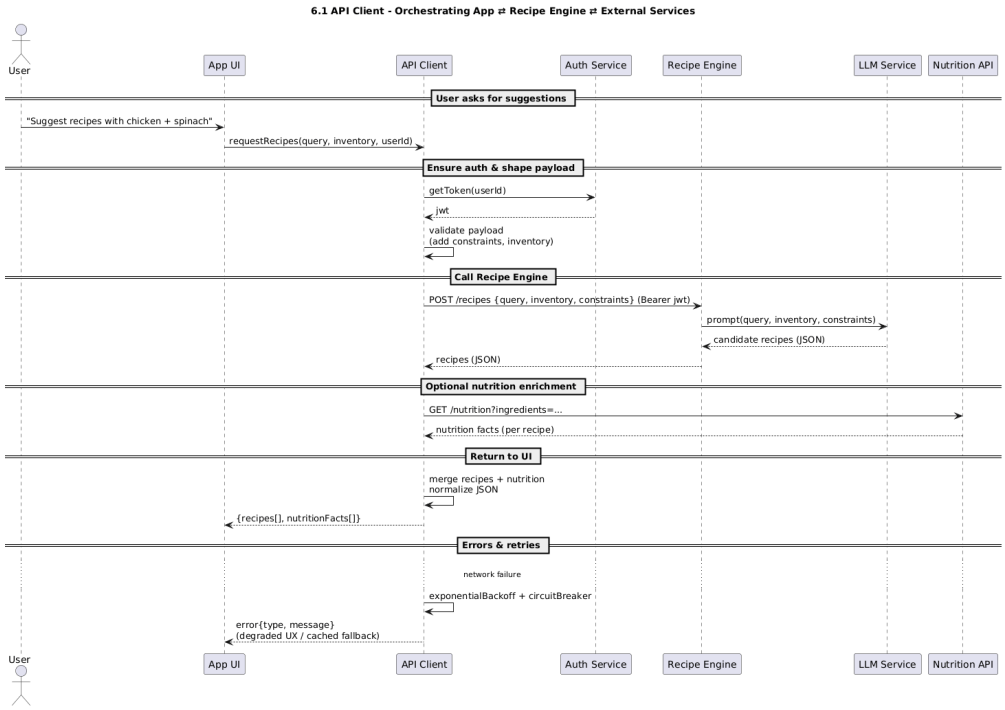

Figure 19: API client and external communication flow

6.2 Local Storage

The Local Storage module ensures that user data and inventory remain accessible even without an internet connection. It stores key information such as personal preferences and food items locally on the device.

Assumptions:

- Uses SQLite or AsyncStorage depending on device support.
- Data must remain secure and lightweight.

Responsibilities:

- Store inventory and user profile locally.
- Enable offline-first app usage.
- Sync with cloud services when online.

Interfaces:

- Inputs: onboarding details, inventory changes.
- Outputs: cached user profile and inventory for the app.

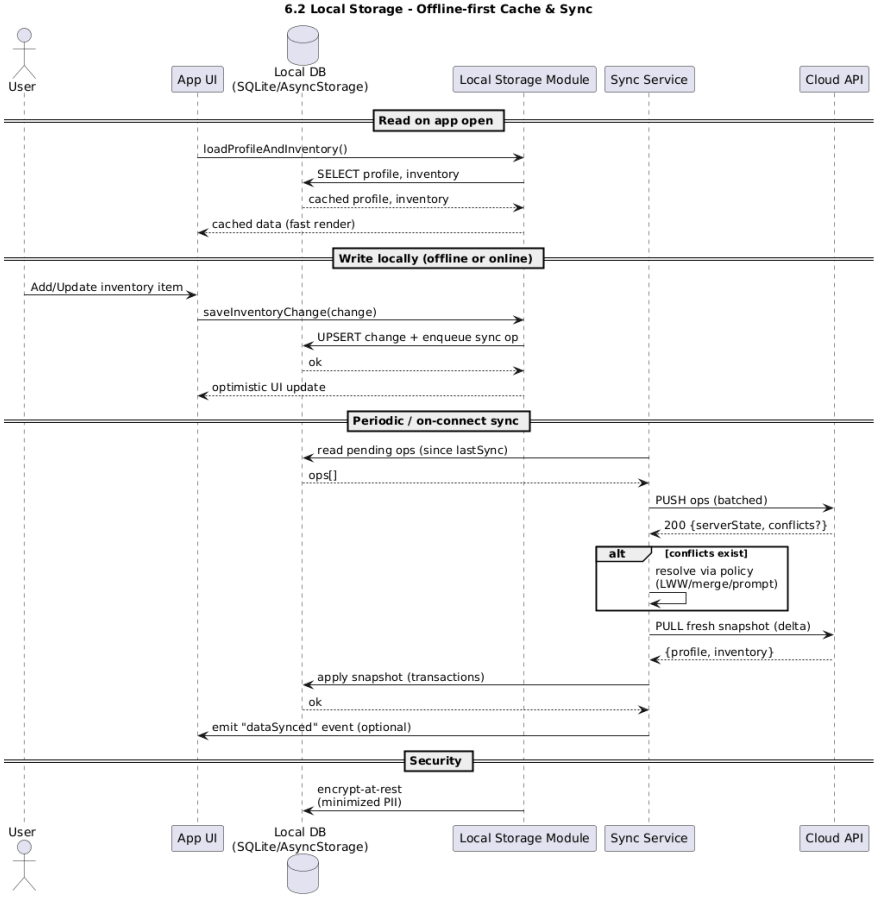

Figure 20: Local storage module for offline-first design

6.3 Push Notification Service

The Push Notification Service manages the delivery of alerts and reminders. It integrates with app logic to notify users about inventory updates, recipe prompts, or personal reminders.

Assumptions:

- Requires device permissions for notifications.
- Notifications must be lightweight and non-intrusive.

Responsibilities:

- Deliver inventory alerts (e.g., expiring items).
- Send recipe-related prompts.
- Trigger custom reminders set by the user.

Interfaces:

- Inputs: events from inventory and recipe engine.
- Outputs: push notifications and in-app alerts.

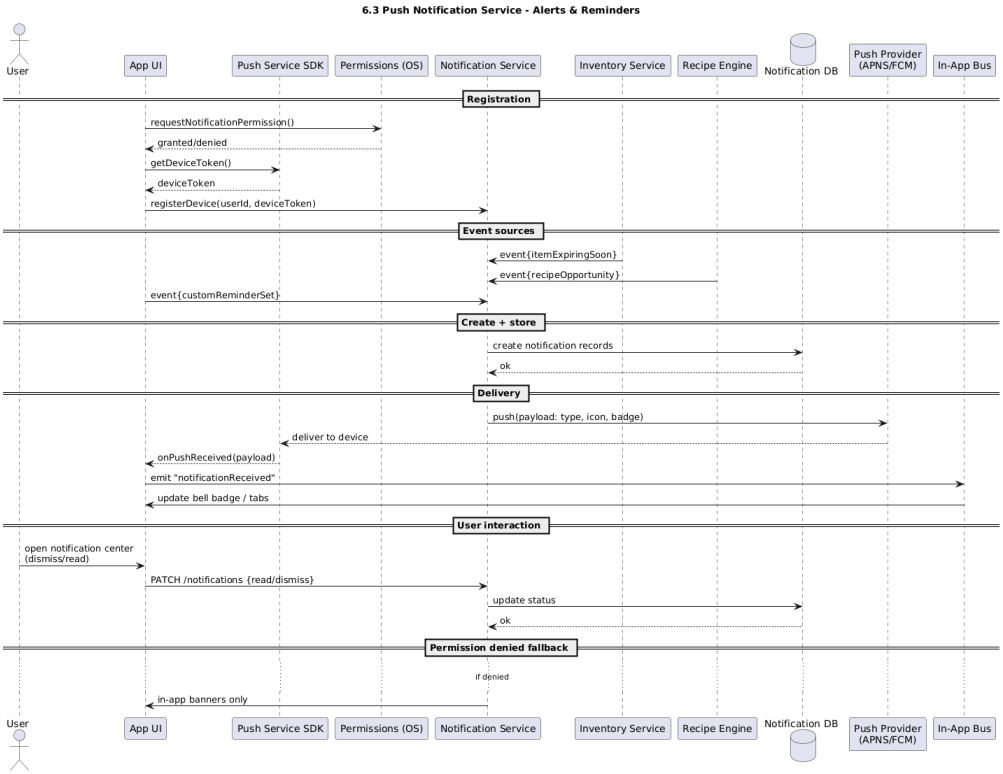

Figure 21: Push notification delivery system

6.4 External Integrations

External integrations extend the functionality of the app by connecting it with third-party services. These include barcode scanning, nutrition databases, and cloud sync across devices.

Assumptions:

- Barcode scanning will require camera and storage permissions.
- Nutrition databases will require API keys and internet access.

Responsibilities:

- Enable barcode/label scanning for easy inventory input.
- Provide detailed nutrition data for recipes.
- Allow seamless syncing across multiple devices in future updates.

Interfaces:

- Inputs: scanned barcode, user query, inventory data.
- Outputs: nutrition facts, auto-filled inventory entries, synced profiles.

REFERENCES